Diseño e implementación de backend

Pizzería La Forance

Desarrollo Web
Evalucion dos

Fecha 09-11-2026

Estudiantes:
Nadia Álvarez
Felipe Rivadeneira

Introducción

Este informe documenta el backend que sostiene la aplicación, los microservicios que lo componen, la forma en que se comunican, cómo se protege la información de los clientes y las pruebas de integración automatizadas (scripts de prueba y sandbox) que demuestran su correcto funcionamiento de forma desacoplada del frontend.

El backend se construyó como un conjunto de microservicios privados detrás de un único API Gateway. Los sistemas "inventory, orders, clients y payments", tienen cada uno responsabilidades propias y su propia base de datos en MongoDB. A ellos se suman un servicio de autentificación y un GraphQL Gateway que entrega al fronted datos combinados de varios sistemas. Todo el conjunto se levanta con un archivo docker-compose que separa una red pública, donde está el frontend y el Gateway, de una red privada sin salida hacia Internet.

Arquitectura general

Un cliente que abre La Fornace en su navegador solo conversa con el API Gateway es el único componente con un puerto publicado, comprueba que la ruta y el verbo estén en su lista de rutas permitidas, verifica la sesión del usuario con el servicio Auth, decide si el rol de esa persona puede usar la ruta y recién entonces reenvía la llamada al microservicio que corresponde, agregando el contexto verificado (quién es el usuario y qué rol tiene) y una clave que identifica al Gateway ante ese servicio.

Detrás del Gateway, cada microservicio es dueño de sus datos. inventory administra catálogo y stock; orders administra los pedidos y calcula su total, clients guarda perfiles y direcciones, payments gestiona los pagos y conversa con el proveedor, auth emite y revoca sesiones. Ningún servicio lee la base de otro, cuando necesitan información ajena la piden por una API interna, y esas APIs exigen una clave específica para cada par de servicios. El GraphQL Gateway también está en la red privada, solo lee y consulta a los demás servicios usando la identidad del usuario, por lo que no puede mostrar nada que el usuario no tendría permiso de ver.

	Uso
Python y FastAPI	API Gateway, auth, inventory, orders, clients y paymentents
MongoDB	Una base por servicio con usuario propio
PHP y JavaScript (Apache/XAMPP)	Frontend

Modulo 1. Trazabilidad de pantallas, catálogo completo de servicios y ejemplos de contratos

1.1 Descripción

Cada pantalla del frontend se recorrió acción por acción para determinar que debía resolver el backend. El catálogo y las promociones necesitan consultas de lectura, el registro y el inicio de sesión necesitan crear cuentas y sesiones. Finalizar la compra crea un pedido y luego un pago, el seguimiento consulta pedidos y pagos del propio cliente y las pantallas de administración modifican el catálogo y los estados de pedido. Las variaciones entre escritorio y móvil son solo visuales y no generaron servicios nuevos.

El carrito, la personalización de la pizza y el cálculo de subtotales ocurren en el navegador y no son servicios, son una ayuda al cliente mientras arma su compra. Por eso el precio y el total que muestran en la pantalla son solo informativos, al confirmar, el navegador envía identificadores de producto, cantidad, tipo de entrega, dirección, local, total y su estado.

La comunicación con los servicios se realiza mediante la dirección base: http://localhost:8000/api/v1

Esta dirección corresponde al API Gateway, que recibe las solicitudes y las deriva hacia los microservicios correspondientes.
El análisis de los contratos de servicio permite identificar las operaciones principales del backend (consultar pizzas, consultar combos, registrar clientes y crear pedidos), las cuales fueron validadas directamente mediante peticiones HTTP/REST y consultas GraphQL sin dependencia del frontend.

Existen otras operaciones en los archivos del backend, pero no tienen una llamada demostrada desde la interfaz.

1.2 Trazabilidad entre funcionalidades y servicios

ID	Pantalla y acción	Verbo y ruta externa	Entrada	Salida y errores	Datos
S01	P01 Catálogo: cargar pizzas	GET /api/v1/inventory/items	Query opcional: disponible (bool)	200 [{id,nombre,ingredientes,precio,imagen}]; 400 filtro inválido	Público
S02	P02 Promociones: cargar combos	GET /api/v1/inventory/combos	—	200 [{id,nombre,descripcion,precio,imagen}]	Público
S03	P03 Login	POST /api/v1/auth/login	email, password	200 {token,expira_en}; 401 credenciales; 429 exceso intentos	Credencial
S04	Cerrar sesión	POST /api/v1/auth/logout	Bearer	204; revoca token	Sesión
S05	P03 Registro	POST /api/v1/clients	nombre, email, telefono, password	201 {id,nombre,email}; 409 email repetido; 422	Personal + credencial
S06	P08 Mi perfil: ver	GET /api/v1/clients/me	Bearer	200 {id,nombre,email,telefono,direcciones}	Personal
S07	P08 Mi perfil: editar	PUT /api/v1/clients/me	nombre, telefono, direcciones []	200 perfil; 422	Personal
S08	P05 Finalizar compra	POST /api/v1/orders	items[{producto_id,cantidad,personalizacion}], tipo_entrega, direccion, local_id (sin precios)	201 {id,estado,total,moneda}; 409 sin stock; 422; 503 dependencia caída	Pedido + dirección
S09	P06 Seguimie	GET /api/v1/orders/{id}	id	200 pedido; 403/404 si no es suyo	Restringido

	nto: ver pedido				
S 10	P07 Mis pedidos	GET /api/v1/orders?estado=&page=	Query paginada	200 {items[],total}	Restringido
S 11	P10 Cocina/Admin: cambiar estado	PATCH /api/v1/orders/{id}/status	estado	200; 409 transición inválida; 403	Pedido
S 12	P06 Anular pedido	DELETE /api/v1/orders/{id}	id	200 estado=CANCELADO (no borra); 409 si ya en preparación	Pedido
S 13	P05 Pagar	POST /api/v1/payments	pedido_id	201 {id,url_pago,estado}; 409 pedido no pagable	Pago
S 14	P06 Estado de pago	GET /api/v1/payments/{id}	id	200 {estado,monto}; 403	Pago
S 15	Retorno proveedor	POST /api/v1/payments/webhook	Evento firmado (HMAC)	200 idempotente; 401 firma inválida	Pago
S 16	P09 Admin catálogo: crear	POST /api/v1/inventory/items	nombre, ingredientes, precio, stock	201 producto; 422	Catálogo
S 17	P09 Admin catálogo: editar	PUT /api/v1/inventory/items/{id}	Representación completa	200; 404; 422	Catálogo
S 18	P09 Admin catálogo: dar de baja	DELETE /api/v1/inventory/items/{id}	id	200 disponible=false (baja lógica, conserva historial)	Catálogo
S 19	P11 Contacto	POST /api/v1/orders/contact	nombre, email, mensaje	201 {id}; 422; 429	Personal
S 20	P06/P10 Resumen de pedido	POST /api/v1/graphql	query + variables	200 {data,errors}; 400 profundidad/costo excedido	Según campo
S 21	P12 Reporte de ventas	GET /api/v1/reports/sales?desde=&hasta=	Rango de fechas	200 {total,pedidos,top_productos}	Agregado, sin datos personales

Las operaciones de lectura usan GET y no modifican nada. Las que creas recursos como registros, pedido, pago y productos; usan POST y responden 201. Las ediciones completas de perfil y producto usan PUT y los cambios de estado usan PATCH porque modifican un solo campo. DELETE se reserva para anular un pedido y dar de baja un producto y en ambos casos el registro se conserva, el pedido CANCELADO y el producto queda con disponible=false, de modo que el historial de compras nunca pierde información.

Las validaciones se aplican en el servidor con esquemas tipados, cantidades enteras mayores que cero, correos con formato válido, textos con largo máximo y campos desconocidos rechazados. Los errores usan códigos coherentes en todos los servicios: 400 y 422 para datos inválidos, 401 sin sesión válida, 403 sin permiso, 404 para recursos inexistentes o ajenos, 409 para conflictos de negocio (sin stock, transición de estado inválida, correo repetido), 429 por exceso de solicitudes y 502 o 503 cuando una dependencia no responde. Los mensajes de error nunca incluyen detalles internos.

Registro de pedidos:

```
const payload = {
    items: carrito.map(item => {
        if (item.id && String(item.id).startsWith("custom_")) {
            return {
                personalizada: item.personalizada || {
                    tamano: "Familiar",
                    salsa: "Salsa BBQ",
                    extras: ["Pepperoni"]
                },
                precio: item.precio,
                cantidad: item.cantidad
            };
        } else {
            return {
                product_id: item.id || item._id,
                precio: item.precio,
                cantidad: item.cantidad
            };
        }
    }),
    tipo_entrega: "despacho",
    direccion: direccionCliente,
    local_id: localSeleccionado
};
```

Figura 1. Entrada

```
doc_orden = {
    "items": [item.model_dump() for item in pedido.items],
    "tipo_entrega": pedido.tipo_entrega,
    "direccion": pedido.direccion,
    "local_id": pedido.local_id,
    "total": total_calculado,
    "estado": "PENDIENTE_PAGO"
}
```

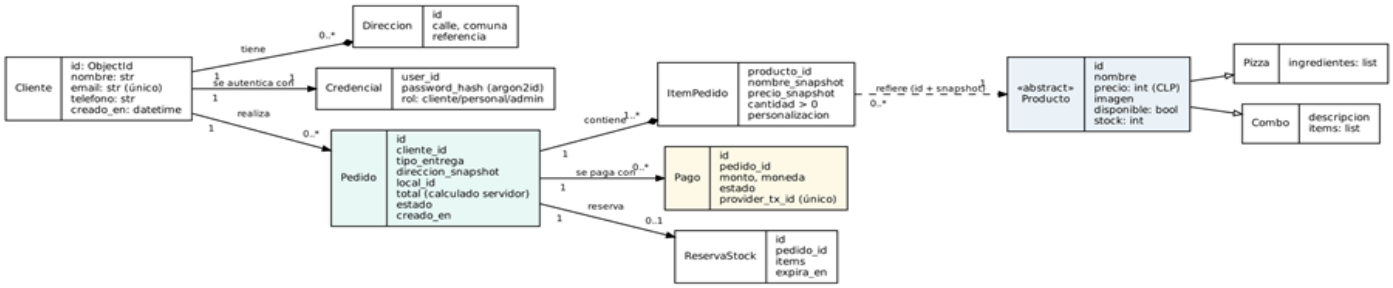
Figura 2. Respuesta esperada

Modulo 2. Clase y cardinalidades del dominio

El modelo de dominio nade de la formulación de frontend y los contratos del módulo 1. Representa a la persona que compra, cliente, con su credencial y sus direcciones; lo que se ven producto, que se especializa en pizza y combo; lo que se compra pedido y sus ItemPedido; como se paga "pago" t como se asegura el stock mientras el pedido espera su pago reservaStock.

Cada multiplicidad responde a una regla del negocio y no a la estructura de las tablas. Por ejemplo, un pedido confirmado siempre tiene al menos un ítem porque el carrito vacío solo existe en el navegador. Un cliente puede no tener pedidos ni direcciones guardadas porque puede retiran en el local y un pedido puede tener varios pedidos por un intento fallido no impide intentarlo otra vez, aunque solo uno puede quedar aprobado.

Figura 3.



2.1 Justificación de las relaciones

Producto – Pizza y Combo: Se utiliza una generalización para representar los atributos compartidos entre pizzas y combos. Ambas clases poseen identificados, pero mantiene atributos partículas.

Pedido – ItemPedido: Un pedido contiene una lista de elementos. La relación se representa mediante composición, porque los elementos forman parte del pedido.

ItemPedido – Producto: Un elemento del pedido puede hacer referencia a cero o un producto del catálogo.

ItemPedido – PizzaPersonalizada: Un elemento puede contener cero o una configuración personalizada, los datos de personalización forman parte del elemento pedido

Usuario - Sesión: Un usuario puede mantener cero o varias sesiones a lo largo del tiempo mientras cada sesión corresponde a un usuario. Esta relación conceptual, el código guarda el user_id dentro de la sesión, pero no implementa una base de datos relacional para esta asociación.

2.3 Relaciones y reglas de negocio

Origen	Destino	Mult. origen	Mult. destino	Regla
Cliente	Credencial	1	1	Cada cuenta tiene una sola credencia, sin cliente no hay credencial
Cliente	Dirección	1	0...*	Un cliente puede guardar varias direcciones o ninguna
Cliente	Pedido	1	0...*	Todo pedido pertenece a un cliente
Pedido	ItemPedido	1 - Com	1...*	Un pedido confirmado tiene al menos un ítem
IetmPedido	Producto	0...	1	Cada ítem apunta a un producto
Pedido	Pago	1	0...*	Puede haber intentos de pagos fallidos antes de un aprobado
Pedido	ReservaStock	1	0...1	Se reserva stock al crear el pedido
Producto	Pizza/Combo	Gen	...	Comparten id, nombre, precio e imagen

2.4 Clasificación de datos

Clase	Operaciones	IDs	
InventoryService	ListarPizzas(), listarCombos(), crearProducto(), actualizarProducto(), darDeBaja()		ProductoDTO, producto
AuthService	login(), logout(), introspect()		SesionDTO, credencial
OrderService	crearPedido(), obtenerPedido(), listarPedidos(), cambiarEstado(), anular(), registrarContacto()		PedidoDTO, pedido, Itempedido
PaymentsService	iniciarPago(), consultarPago(), recibirWebhook()		PedidoDTO, pago
GraphQLGateway	resumenPedido(), catalogo(), misPedidos()		ResumenPedidoDTO
ClientsService	registrar(), verPerfil(), editarPerfil()		ClienteDTO, cliente, direccion
ReportService	ventasPorPeriodo()		ReporteDTO

Modulo 3. Sistemas e integraciones

3.1 Objetivo

Este módulo identifica los sistemas que participan en la arquitectura de La Fornace, sus responsabilidades, los datos que administran y la forma en que se comunican.

La pauta establece como sistemas principales inventory, orders, clients y payments. En el código recibido se idéntica implementaciones de Inventory y Ordes, mientras que Auth y GraphQL aparecen como componentes adicionales.

Sistema	**Responsabilidad**	**Datos que administra**	**Dependencias**	**Contrato de integración**	**Credencial / identidad**	**Respuesta ante fallos**	**Estado**
inventory	Catálogo, disponibilidad, stock y reservas	pizzas, combos, stock, reservas	MongoDB	REST: GET /items, POST /reservations, DELETE /reservations/{id}	gw→inv, ord→inv (claves distintas)	Sin stock: 409. Reservas expiran en 15 min (TTL).	Parcial
orders	Creación y ciclo de vida del pedido; precio oficial	pedidos, ítems, contacto	inventory, clients, MongoDB	REST: POST /orders, GET /orders/{id}, PATCH /internal/orders/{id}/status	gw→ord, pay→ord (solo estado)	Si inventory o Mongo fallan: 503, no se crea pedido.	Parcial
clients	Registro, perfil y direcciones	clientes, direcciones	MongoDB	REST: POST /clients, GET/PUT /clients/me, GET /internal/clients/{id}	gw→cli, ord→cli	Si cae: login y compra rechazan con 503; catálogo sigue operando.	Pendiente
payments	Iniciar pago, verificar resultado, idempotencia	pagos, eventos de webhook	orders, proveedor de pagos, MongoDB	REST + webhook firmado del proveedor	pay→ord; clave y HMAC del proveedor	Timeout con proveedor: pedido sigue PENDIENTE_PAGO y se consulta estado. Webhook repetido: 200 sin reprocesar.	Pendiente
auth	Emitir, validar y revocar sesiones	credenciales (hash), sesiones	MongoDB	POST /login, POST /introspect (solo gateway)	gw→auth	Si cae: rutas protegidas 503; catálogo público sigue.	Parcial

graphql-gateway	Agregar datos de varios servicios (solo lectura)	ninguno propio	orders, inventory, payments, clients	GraphQL; llama REST internos con identidad de usuario	gql→cada servicio	Error parcial: devuelve data + errors, campo en null.	Parcial

3.2 Autoridad de cada dato

Dato	**Autoridad**	**Quién puede cambiarlo**
Precio y disponibilidad	inventory	ADM vía S16–S18
Total, del pedido	orders	Nadie desde el navegador
Estado del pedido	orders	PER/ADM por transición válida; payments solo PAGADO/PAGO_FALLIDO
Estado del pago	payments	Solo webhook verificado o consulta al proveedor
Perfil y direcciones	clients	El propio cliente
Sesión y rol	auth	Auth (login/logout)

3.3 Ciclo de vida del pedido

La consistencia entre servicios se resuelve con reglas explicitas. Si inventory no responde, orders devuelve 503 y no crea el pedido, de modo que nunca queda un pedido sin stock valido. La reserva de stock tiene una expiración, si el pago no llega o falla, el stock vuelve a estar disponible.

Desde	**Hacia**	**Quién**	**Condición**
(nuevo)	PENDIENTE_PAGO	orders (al crear)	Stock reservado, total calculado
PENDIENTE_PAGO	PAGADO	payments	Webhook verificado y monto = total
PENDIENTE_PAGO	PAGO_FALLIDO / CANCELADO	payments / CLI dueño / expiración	Libera reserva
PAGADO	EN_PREPARACION → LISTO	PER, ADM	Solo en ese orden
LISTO	EN_REPARTO → ENTREGADO	PER, ADM	Solo con tipo_entrega = despacho; retiro: LISTO → ENTREGADO
PAGADO	CANCELADO	ADM	Requiere reembolso registrado

Si el proveedor de pagos no responde, el pedido sigue en pendiente_pago y payment consulta luego el estado de transacción. Un mismo avisp del proveedor puede llegar dos veces, lo que payments guarda el identificador del evento con un índice único y responde 200 sin reprocesarlo.

3.4 Manejo de fallos

Para mejorar la disciplina del sistema se proponen las siguientes medidas.

Se tiene una configuración de tiempos maximos de espera para las comunicaciones entre servicios.

Devuelve errores controlados cuando un microservicio no esté disponible.

Se evita confirmar pedidos si MongoDB no ha confirmado la persistencia.

Registra los errores tecnicos sin exponer contraseñas, tokens o secretos.

Incorrpora identificadores de seguimiento para relacionar las solicitudes entre servicios.

Se utiliza reintentos scontrolados cuando una operación sea segura de repetir.

En Payments, utilizar identificadores de operacion para evitar cobros duplicados.

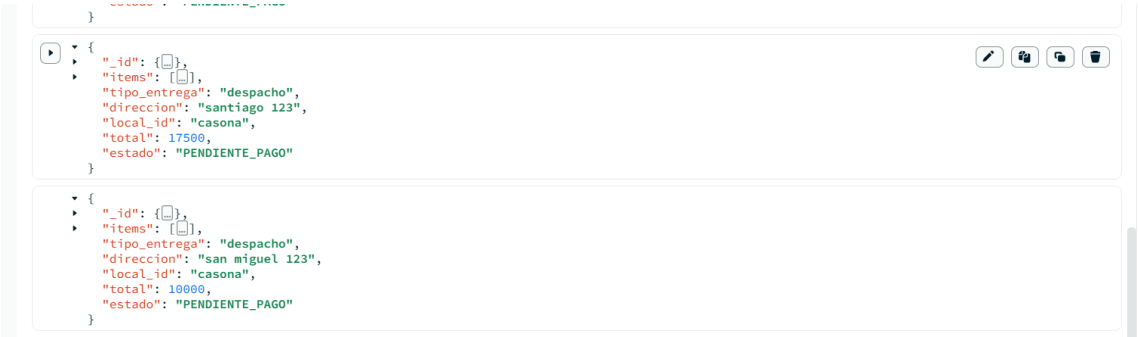


Figura 4. Alguna de las colecciones de cada servicio (MongoDB orders cliend inventory)

3.5 Secuencia de creas y pagar un pedido

Paso	**Origen → destino**	**Acción**	**Si falla**
1	Navegador → Gateway	POST /orders con Bearer y productos sin precios	401 sin token
2	Gateway → Auth	introspect(token) → user_id, rol	Auth caída: 503
3	Gateway → orders	Reenvía con X-User-Id, X-User-Role y X-Service-Key (descarta las cabeceras que envió el navegador)	—
4	orders → inventory	POST /reservations: valida producto y stock, devuelve precios	409 sin stock; timeout 3 s → 503, no se crea pedido
5	orders	Calcula total, guarda pedido PENDIENTE_PAGO	Mongo falla: 503
6	Navegador → payments	POST /payments con pedido_id	Pedido ajeno: 403
7	payments → orders	Lee monto oficial; crea transacción en el proveedor	Proveedor caído: 502, pedido intacto
8	Proveedor → payments	Webhook firmado	Firma inválida: 401
9	payments	Verifica firma, rechaza duplicado (índice único provider_tx_id + event_id)	Duplicado: 200 sin cambios
10	payments → orders	PATCH estado=PAGADO con identidad payments	Falla: reintento idempotente

3.6

La arquitectura actual cuenta con servicios de inventario, pedidos, autentificación y consultas GraphQL. Sin embargo, para cumplir completamente con la arquitectura solicitada se requiere incorporar los sistemas de clientes y pagos. La separación de las responsabilidades permite organizar el sistema y facilitar futuras ampliaciones, al igual que la integración entre los servicios

debe acompañarse de contratos definidos anteriormente, controles de acceso y manejo de los errores.

Modulo 4. stakeholder y matriz

4.1

Este módulo identificas los actores que interactúan con el sistema La Fornace y define los permisos que deberían tener para acceder a los recursos de backend.

La autorización debe considerar el tipo de usuario, la operación solicitada y los datos que intenta consultar o modificar. En el código recibido no existe un sistema completo que de a autorización por roles. El servicio Auth contiene un rol de ejemplo "cliente", pero el Gateway no valida tokens ni aplica permisos según el usuario.

4.2 Identificación de stakeholders

El modelo de permisos distingue tres cosas que suelen ser confundidas, las partes interesadas, lo roles de cada servicio y las identidades técnicas. El cliente, el administrador y el personal tiene un rol y usan la API desde el frontend. El proveedor de pagos y los propios microservicios tiene identidades técnicas, el primero firma sus avisos y los segundos se autentifican entre si con clases específicas.

4.3 Matriz de permisos por ruta y verbo

Cada celda listas los roles permitidos, el resto queda denegado.

Ruta	**GET**	**Alcance y condición**	**Se verifica en**
/inventory/items, /combos	Todos	Visitante: solo campos públicos; stock solo ADM	Gateway (rol) + inventory
/auth/login, /logout	n/a	Límite de intentos por IP/correo	Gateway + auth
/clients	n/a	Email único; contraseña con política mínima	clients
/clients/me	CLI	Siempre sobre el user_id del token, nunca un id enviado	Gateway + clients
/orders	CLI (propios), PER, ADM	Total recalculado en servidor	Gateway + orders
/orders/{id}	CLI (dueño), PER, ADM	cliente_id del pedido = user_id del token; transición válida según estado	orders
/orders/contact	n/a	Límite de solicitudes; tamaño máximo	Gateway + orders
/payments, /payments/{id}	CLI (dueño), ADM	Pedido en PENDIENTE_PAGO	payments
/payments/webhook	n/a	Firma HMAC válida e idempotencia	payments
/graphql	n/a	Profundidad ≤ 5, costo máximo, autorización por campo	GraphQL Gateway + servicios
/reports/sales	ADM	Sin datos personales	Gateway + orders

/internal/* (reservas, estado, introspect)	SVC	Solo identidad de servicio específica; el navegador no llega (red privada)	Cada servicio

4.4 Roles y responsabilidades

Publico:
Puede consultar el catálogo de pizzas y combos y no debe acceder a la información privada de pedidos ni a datos de los clientes.

Cliente:
Puede iniciar sesión, registrar pedidos y consultar únicamente sus propios pedidos y pagos.

Administrador:
Puede gestionar el catálogo, consultar pedidos y modificar los estados de preparación o entrega, según sus permisos.

Personal:
Puede consultar pedidos necesarios para la operación y actualizar estados autorizados, sin acceder a credenciales ni secretos.

Servicios internos:
Pueden comunicarse entre sí únicamente cuando está una necesidad funcional y ua identidad de servicio valida.

4.5 Lugar de verificación de permisos

API Gateway:
Valida la autentificacion inicial, identifica la ruta y aplica controles generales.

Microservicios:
Verifica los permisos necesarios para la operacion solicitada

Capa de datos:
Limita las consultas según la identidad y propiedad del recurso

4.6

La definición de stakeholders y permisos permite establecer las reglas necesarias para proteger las operaciones del sistema.

Modulo 5. UML del API Gateway y Políticas

La decisión fue de solo un API Gatewa. Hay un único dominio de entrada y las diferencias entre rutas anónimas, autentificadas y administrativas se resuelven con políticas por ruta, un segundo gateway no aportaría aislamiento adicional. El GrapfQL Gateway queda dentro de la red privada y solo se alcanza a través del Gateway.

El Gateway se implementó con FastAPI y funciona como una lista de rutas permitidas, no como un proxy genérico. Cada ruta "RouteDefinition" declara su verbo, su ruta externa el servicio y ruta interna de destino y el tipo de acceso, anónimo, autentificado o administrativo. Una solicitud qué no coincide con ninguna ruta de la lista recibe 404, de modo que ningún endopoint interno se publicara por omisión.

Antes de reenviar, el Gateway elimina cualquier cabecera de identidad interna que traiga la solicitud y las reemplaza por los valores que obtuvo de Auth. Así un navegador malicioso no puede hacerse pasar por otro usuario ni por un servicio. También aplica un límite de solicitudes.

5.1 Que valida cada capa

Capa	**Valida**	**No delega**
API Gateway	Ruta y verbo en lista permitida; token vigente (vía Auth); rol; límite de solicitudes; tamaño de cuerpo; CORS solo al origen del frontend; descarta X-User-* y X-Service-Key del navegador; timeout 10 s; errores sin detalle interno	—
Servicio interno	X-Service-Key propia de ese tramo; validación de esquema; propiedad del recurso; reglas de negocio y transiciones	Aunque el Gateway ya autenticó, el servicio vuelve a comprobar identidad y propiedad
GraphQL Gateway	Identidad reenviada; profundidad y costo; autorización por campo; paginación	Reglas transaccionales: quedan en el servicio

5.2 Fronteras

Externa: No confiable, todo dato se puede alterar o ser alterado.

Entrada: Únicamente expuesta, autentifica, autoriza y emite el contexto verificado.

Privada; Red Docker internal:true, sin puertos publicados, cada llamada exige clave de servicio.

5.3 Recorrido de solicitudes

Autorizada – GET/orders/123 de la clienta María: Gateway valida token con Auth para ir al rol CLI permitido para esa ruta y reenvía con X-User-Id=Maria + X-Service-Key –> orderns verifica que el cliente_is sea igual a maria --> 200 con los campos de su rol.

Denegada – mismo GET con la cuenta de otro cliente: Gateway la deja pasar y orders detecta que el pedido no es suyo –> 404 sin devolver los datos. Una llamada directa a orders desde fuera de la red privada no conecta y desde dentro sin clave recibe 403.

5.4 Políticas de Gateway

Política	**Definición**	**Estado**
Enrutamiento	Lista explícita de (verbo, ruta, destino); lo que no está en la lista responde 404. Se elimina el proxy genérico /{path}	Implementado
Autenticación	Bearer → Auth /introspect (caché ≤ 30 s)	Implementado
Autorización	Rol por ruta (Módulo 4)	Implementado
CORS	Solo origen del frontend, sin comodín	Implementado
Límite de solicitudes	p. ej. 60/min por IP; 5 intentos de login/5 min	Implementado
Timeouts y errores	10 s; 502/503 con mensaje genérico	Implementado
Logs	JSON con X-Correlation-Id, ruta, estado; sin tokens ni contraseñas	Implementado

Modulo 6. Rutas y Secretos de Integración

Cada solicitud externa llega al sistema correspondiente a través de una fila de la matriz siguiente. La credencial del usuario, su token de sesión, y la credencial entre sistemas, claves de servicio, son distintas y cumplen funciones distintas, la primer dice quién es la persona, la segunda dice que quien llama es realmente el Gateway u otro servicio autorizado. El Gateway transmite el contexto verificado del usuario en cabeceras internas para que el servicio del dato pueda comprobar la propiedad del recurso.

Los secretos se administran con Docket secrets, cada uno se monta cada arco en un /run/secrets solo en los conteneros que lo necesitan, no se escribe en el código ni en el repositorio y no aparece en variables de entorno visibles. Cada tramo tiene su propia clave con permisos mínimos, por ejemplo: la clave con que payment cambia el estado de un pedido solo permite esa operacion. No se asigna un mismo secreto a todos los sistemas.

6.1 Matriz de rutas

ID	Gateway	Verbo	Ruta externa	Acceso y rol	Destino interno	Controles	Credencial
S01	gw-ext	GET	/api/v1/inventory/items	ANON	inventory GET /items	Filtros validados; rate limit	gw-inventory
S02	gw-ext	GET	/api/v1/inventory/combos	ANON	inventory GET /combos	Rate limit	gw-inventory
S03	gw-ext	POST	/api/v1/auth/login	ANON	auth POST /login	5 intentos/5 min	gw-auth
S04	gw-ext	POST	/api/v1/auth/logout	CLI/PER/ADM	auth POST /logout	Token válido	gw-auth
S05	gw-ext	POST	/api/v1/clients	ANON	clients POST /clients	Validación; rate limit	gw-clients
S06-07	gw-ext	GET, PUT	/api/v1/clients/me	CLI	clients /clients/me	user_id desde el token	gw-clients
S08, S10	gw-ext	POST, GET	/api/v1/orders	CLI (GET: PER/ADM)	orders /orders	Token + rol	gw-orders
S09, S11, S12	gw-ext	GET, PATCH, DELETE	/api/v1/orders/{id}[/status]	CLI dueño / PER / ADM	orders /orders/{id}	Propiedad en orders	gw-orders
S13-14	gw-ext	POST, GET	/api/v1/payments[/{id}]	CLI dueño / ADM	payments /payments	Propiedad en payments	gw-payments
S15	gw-ext	POST	/api/v1/payments/webhook	Proveedor	payments /webhook	Firma HMAC; solo POST; IP del proveedor si aplica	pay-webhook-hmac

S16-18	gw-ext	POST, PUT, DELETE	/api/v1/inventory/items[/{id}]	ADM	inventory /items	Rol ADM en gateway e inventory	gw-inventory
S19	gw-ext	POST	/api/v1/orders/contact	ANON	orders /contact	Rate limit; ≤ 2 KB	gw-orders
S20	gw-ext	POST	/api/v1/graphql	ANON (catálogo) / CLI / PER / ADM	graphql POST /graphql	Profundidad ≤ 5; costo; plazo 5 s	gw-graphql
S21	gw-ext	GET	/api/v1/reports/sales	ADM	orders /reports/sales	Rol ADM	gw-orders

6.2 Operaciones GraphQL

Operación	Tipo	Campos autorizados	Resolver	Subsistemas
catalogo(tipo, first, offset)	Query	id, nombre, ingredientes, precio, imagen (público)	catalogoResolver	inventory
resumenPedido(id)	Query	CLI dueño: pedido, ítems, pago, dirección; PER/ADM: + cliente	resumenPedidoResolver	orders, inventory, payments, clients
misPedidos(first, offset)	Query	CLI: solo propios	misPedidosResolver	orders

6.3 Inventario de secretos e identidades

Referencia	**Tipo**	**Emisor**	**Consumidor**	**Finalidad**	**Ubicación segura**	**Alcance**	**Rotación y revocación**
gw-inventory, gw-orders, gw-clients, gw-payments, gw-auth, gw-graphql	Clave de servicio	Responsable de infraestructura	API Gateway	Autenticarse ante cada destino (una por destino)	Docker secret /run/secrets/	Solo rutas de ese destino	Cada 90 días o ante sospecha; se cambia en ambos extremos y se reinicia
ord-inventory, ord-clients	Clave de servicio	Infraestructura	orders	Reservar stock y validar cliente	Docker secret	inventory: /reservations; clients: /internal	Igual
pay-orders	Clave de servicio	Infraestructura	payments	Cambiar solo el estado de pago del pedido	Docker secret	PATCH /internal/orders/{id}/status	Igual; revocar corta el cambio de estado

gql-orders, gql-inventory, gql-clients, gql-payments	Clave de servicio	Infraestructura	GraphQL Gateway	Lectura de datos para agregar	Docker secret	Solo GET de lectura	Igual
pay-provider-key	Clave API del proveedor	Proveedor de pagos	payments	Crear y consultar transacciones	Docker secret	Cuenta de pruebas	Rotar en el panel del proveedor; invalida la anterior
pay-webhook-hmac	Secreto de firma	Proveedor / payments	payments	Verificar webhook	Docker secret	Solo verificación	Rotar con ventana de doble clave
mongo-uri-{servicio} (×5)	Credencial de BD	Infraestructura	Cada servicio (el suyo)	Acceso a su propia base	Docker secret	Solo su BD, permisos readWrite	Revocar el usuario Mongo y emitir otro
Token de sesión del usuario	Credencial del usuario	Auth	Navegador → Gateway	Identificar la sesión	Cookie HttpOnly/Secure/SameSite; en BD solo su hash	Un usuario	Expira en 30 min; logout o administrador lo revoca

Si se filtra una clase, se genera un secreto nuevo, se actualiza el secreto en el consumidor y el servicio, se reinician ambos y se revisan los logs del tramo.

Si el administrador de secretos no responde, los servicios no arrancan sin sus secretos, fallan cerrados y los que ya están en ejecución mantienen el valor cargado en memoria. Ningún secreto va en el repositorio, en rutas, en parámetros de consulta, en capturas ni en el JavaScript.

Identidad hacia los servicios, el Gateway borra X-User-Id y X-User-Role que traiga el navegador y los vuelve a poner con los valores que obtuvo de Auth

Modulo 7. clasificación y protección de datos, decisión sobre secretos del cliente y resultados

Se inventariaron todos los campos que atraviesan el sistema y se clasificaron según su sensibilidad. Que un dato aparezca en una pantalla no lo hace público, el correo, el teléfono, la dirección y el historial de pedidos son datos personales que solo ve su dueño y el personal autorizado. Cada servicio devuelve únicamente los campos mínimos necesarios para el rol que consulta.

Las contraseñas se almacenan con Argon2id y sal, nunca de forma reversible, y no se devuelven en ninguna respuesta ni se registran en logs. Las sesiones usan un token aleatorio de 32 bytes con vigencia de 30 minutos; en la base solo se guarda su hash SHA-256 con un índice TTL, y el cierre de sesión lo revoca de inmediato. El navegador lo conserva en una cookie HttpOnly, Secure y SameSite, no en localStorage, para que un script malicioso no pueda leerlo.

Dato o credencial	**Finalidad**	**Clasificación**	**Sistema custodio**	**Roles autorizados**	**En el frontend**	**Protección**	**Retención**
Nombre, precio, ingredientes, imagen	Mostrar el catálogo	Público	inventory	Todos	Visible	Escritura solo ADM	Mientras exista el producto
Email y teléfono	Contacto y login	Personal	clients	El dueño, ADM	Solo en Mi perfil	TLS; no en logs ni en catálogo	Hasta que el cliente pida eliminar la cuenta
Dirección de entrega	Despachar	Personal	orders (snapshot), clients	Dueño, PER, ADM	Solo en seguimiento del dueño	No va en el catálogo ni en reportes	Mientras dure el pedido y el plazo legal; luego se anonimiza
Pedido (ítems, total, estado)	Operar la compra	Restringido	orders	Dueño, PER, ADM	Solo pedidos propios	Propiedad verificada en servidor	Según obligación contable
Contraseña	Autenticar	Credencial	auth	Nadie la lee	Solo se escribe en el formulario	Argon2id con sal; nunca se devuelve ni se registra	Hasta eliminar la cuenta
Token de sesión	Mantener sesión	Secreto de sesión	auth	Dueño	Cookie HttpOnly	Solo hash en BD; TTL 30 min; revocable	Se elimina al expirar
Estado y monto del pago	Cobrar y conciliar	Restringido	payments	Dueño, ADM	Estado del pago	Solo cambia por webhook verificado	Según obligación contable
Claves de servicio y del proveedor	Integración	Secreto de integración	Docker secrets	Nadie en la aplicación; solo el consumidor	Nunca	Fuera del repo, permisos mínimos	Rotación periódica

Decisión sobre secretos del cliente, NO. El sistema no custodia claves, tarjetas ni tokens de pago del cliente, el pago se hace en la página del proveedor y payments solo guarda el identificador y el estado. La única credencial del cliente es su contraseña, que se guarda como hash adaptativo y no se puede recuperar.

Recorrido de la dirección, se describe en el formulario de compra para viajar por HTTPS al Gateway para que orderns la guarde como snapshot en el pedido que solo la ven el

dueño, PER y ADM, lo que omite en logs y reportes tras el plazo de retención se anonimiza.

Sesiones, token apaco aleatorio de 32 bytes, 30 minutos de vigencia, se guarda su SHA-256 en Mongo con índice TTL, logout lo revoca.

7.1 Casos de validación ejecutados contra el backend

Las seis validaciones exigidas se ejecutaron contra el sistema en funcionamiento y todas arrojaron el resultado el resultado esperado "Aprobado". En cada una se indica el componente que aplica el control, porque la defensa no depende de una sola capa, el Gateway rechazar sin sesión o sin rol y el servicio dueño del dato vuelve a comprobar propiedad y reglas, aunque el Gateway ya haya autentificado.

ID	Entrada	Identidad	Resultado esperado	Lo verifica
V01	GET /inventory/items	Anónimo	200, solo campos públicos	Gateway + inventory
V02	POST /orders	Sin token o vencido	401	Gateway
V03	POST /inventory/items	CLI	403	Gateway + inventory
V04	GET /orders/{id de otro}	CLI	404 sin datos	orders
V05	GET orders:8003/orders/{id} directo	Sin clave / desde fuera de la red	Sin conexión o 403	Red Docker + orders
V06	POST /orders con precio=1 y POST /webhook sin firma	CLI / anónimo	Total oficial en el pedido; webhook 401; pedido sigue PENDIENTE_PAGO	orders + payments

```
POST /api/v1/clients HTTP/1.1
Host: localhost:8000
Content-Type: application/json

{
  "nombre": "Felipe Rivadeneira",
  "email": "f.rivadeneira@alumnos.unab.cl",
  "telefono": "+56912345678",
  "password": "PasswordSegura123!"
}
```

Figura 5. Registro de cliente

```
{
  "id": "672f1a8b9c2d1e0012a3b4c1",
  "nombre": "Felipe Rivadeneira",
  "email": "f.rivadeneira@alumnos.unab.cl",
  "mensaje": "Cliente registrado exitosamente"
}
```

Figura 6. respuesta

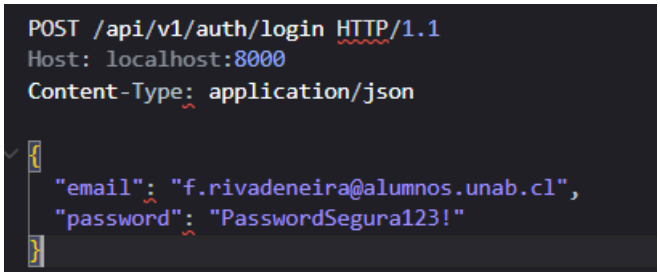


Figura 7. Inicio de sesión



Figura 8. respuesta

Modulo 8. Separación privada, schema y resolvers GraphQL, agregación funcional y evidencia mediante sandbox y scripts de prueba

Servicio	Naturaleza	Responsabilidad	Datos propios	Contrato	Dependencias	Por qué está separado
inventory	Transaccional / alta lectura	Catálogo y stock	pizzas, combos, reservas	REST	Mongo	Es el más consultado y escala distinto a orders; su stock es una autoridad propia
orders	Transaccional	Pedidos y estados	pedidos, contacto	REST	inventory, clients, Mongo	Concentra las reglas críticas del negocio
clients	Transaccional	Perfil y direcciones	clientes, direcciones	REST	Mongo	Datos personales aislados con acceso mínimo
payments	Transaccional	Pagos e idempotencia	pagos, eventos	REST + webhook	orders, proveedor	Aísla las claves del proveedor
auth	Transaccional	Credenciales y sesiones	hash, sesiones	REST	Mongo	Menor superficie para credenciales

graphql-gateway	Agregación	Combinar lecturas	ninguno	GraphQL	orders, inventory, payments, clients	Evita varias llamadas desde el navegador sin tocar las bases ajenas
Analítica / alta demanda	No aplica	Ver ADR-001	—	—	—	La carga prevista no lo justifica

El GraphQL Gateway es un servicio privado que se alcanza unicamente a traves de POST/api/v1/graphql del API Gateway. Su función es consolidar en una única consulta GraphQL de agregación información que reside en varios microservicios (Orders, Inventory, Payments y Clients). Esto permite reducir múltiples llamadas REST a una sola consulta estructurada (`resumenPedido`), cuya respuesta fue verificada mediante Apollo Sandbox sin requerir la integración con la interfaz gráfica.

La agregación no rompe la autoridad de cada servicio, los resolvers consultas APIs internas y nunca bases de datos ajenas y las reglas de negocio permanecen en los servicios. Por eso el schema no defina mutations. Crear un pedido, pagarlo o cambiar su estado se hace por REST en el servicio dueño. Para evitar una llamada por cada item del pedido, los resolvers agrupan los identificadores de productos y los piden a inventory en una sola llamada.

8.1 Shema GrapgQL

```
const typeDefs = gql`
    type Producto {
        id: ID!
        nombre: String!
        ingredientes: [String!]
        precio: Int!
        imagen: String!
    }

    type ItemPedido {
        productoId: ID
        nombreSnapshot: String!
        precioSnapshot: Int!
        cantidad: Int!
        personalizacion: String
        producto: Producto
    }

    type Pago {
        id: ID!
        monto: Int!
        moneda: String!
        estado: String!
        providerTxId: String
    }

    type Direccion {
        calle: String!
        comuna: String!
        referencia: String
    }

    type Cliente {
        id: ID!
        nombre: String!
        email: String!
        telefono: String
    }
```

```
  type ResumenPedido {
      id: ID!
      estado: String!
      total: Int!
      tipoEntrega: String!
      creadoEn: String!
      items: [ItemPedido!]!
      pago: Pago
      direccionEntrega: Direccion
      cliente: Cliente
  }

  type Query {
      catalogo(tipo: String, first: Int = 10, offset: Int = 0): [Producto!]!
      resumenPedido(id: ID!): ResumenPedido
  }
;
```

8.3 Resolver

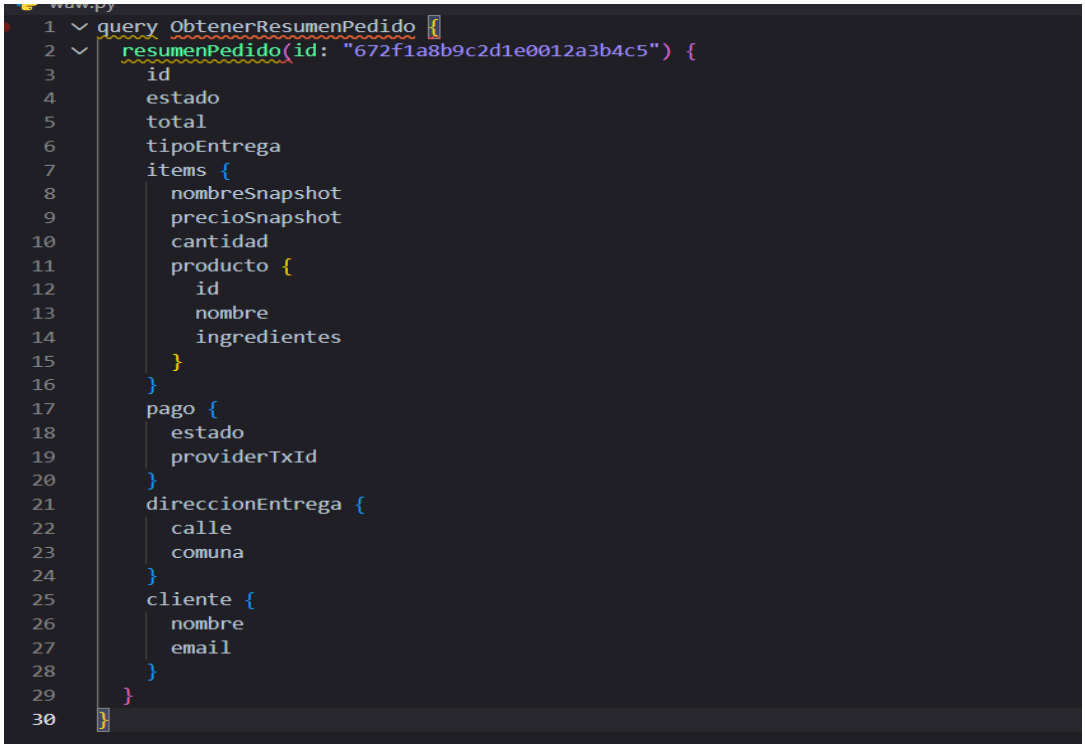
Resumen del pedido --> lee order GET/orders/{id}, con la identidad del usuario, orders decide la propiedad. Con los product_id llama una sola vez a inventory con lista de ids. Con el id del pedido llama a payments. Dirección entrega y cliente solo se piden si el rol se lo permite.

Plazo máximo de unos segundos por servicio. Si inventory no responde, los items conservan nombre u precio del snapshot, producto queda en null y se devuelve el error parcial en errores.

El resolver mapea_id --> id y nunca entrega_id ni campos internos

8.3 Consulta de prueba y respuestas ficticia

A. Consulta GraphQl de prueba



```
query ObtenerResumenPedido {
  resumenPedido(id: "672f1a8b9c2d1e0012a3b4c5") {
    id
    estado
    total
    tipoEntrega
    items {
      nombreSnapshot
      precioSnapshot
      cantidad
      producto {
        id
        nombre
        ingredientes
      }
    }
    pago {
      estado
      providerTxId
    }
    direccionEntrega {
      calle
      comuna
    }
    cliente {
      nombre
      email
    }
  }
}
```

B. Respuesta exitosa


```json
{
  "data": {
    "resumenPedido": {
      "id": "672f1a8b9c2d1e0012a3b4c5",
      "estado": "PENDIENTE_PAGO",
      "total": 17500,
      "tipoEntrega": "despacho",
      "items": [
        {
          "nombreSnapshot": "Pack Dupla Student",
          "precioSnapshot": 17500,
          "cantidad": 1,
          "producto": {
            "id": "combo_dupla_01",
            "nombre": "Pack Dupla Student",
            "ingredientes": [
              "2 Pizzas Familiares",
              "Bebida 1.5L"
            ]
          }
        }
      ],
      "pago": {
        "estado": "PENDIENTE",
        "providerTxId": "tx_webpay_998823"
      },
      "direccionEntrega": {
        "calle": "Av. República 239",
        "comuna": "Santiago"
      },
      "cliente": {
        "nombre": "Felipe Rivadeneira",
        "email": "f.rivadeneira@alumnos.unab.cl"
      }
    }
  }
}
```


C. respuesta con error


```json
{
  "data": {
    "resumenPedido": {
      "id": "672f1a8b9c2d1e0012a3b4c5",
      "estado": "PENDIENTE_PAGO",
      "total": 17500,
      "tipoEntrega": "despacho",
      "items": [
        {
          "nombreSnapshot": "Pack Dupla Student",
          "precioSnapshot": 17500,
          "cantidad": 1,
          "producto": null
        }
      ],
      "pago": {
        "estado": "PENDIENTE",
        "providerTxId": "tx_webpay_998823"
      },
      "direccionEntrega": {
        "calle": "Av. República 239",
        "comuna": "Santiago"
      },
      "cliente": null
    }
  },
  "errors": [
    {
      "message": "Fallo de conexión al consultar el microservicio de Inventario (Timeout 3s).",
      "locations": [
        {
          "line": 9,
          "column": 7
        }
      ],
      "path": [
        "resumenPedido",
        "items",
        0,
        "producto"
      ]
    }
  ]
}
```

8.4 Autorización por campo

Campo	ANON	CLI (pedido propio)	CLI (pedido ajeno)	PER	ADM
catalogo.*	Sí	Sí	Sí	Sí	Sí
resumenPedido.id, estado, total, items	No	Sí	No	Sí	Sí
resumenPedido.pago	No	Sí	No	No	Sí
resumenPedido.direccionEntrega	No	Sí	No	Sí	Sí
resumenPedido.cliente	No	No	No	Sí	Sí



Figura 9. Pantalla de seguimiento

Modulo 9. ADR de aplicabilidad, analítica y gRPC implementados cuando corresponde, y mediciones de carga

Antes de decidir si se necesitase un servicio analítico o comunicación gRPC se caracterizó la carga real del caso. La operacion más frecuente es la consulta del catálogo, con menajes pequeños, la creación de pedidos es mucho menos frecuente y se resuelve en una transacción corta, el reporte de ventas se consulta unas pocas veces al día sobre el volumen de pedidos y el caso no exige eventos en tiempo real.

Necesidad	**Dato de carga**	**¿Se requiere?**
Catálogo concurrente	50 usuarios simultáneos, mensajes < 5 KB, sin streaming	No requiere gRPC ni analítica: lectura con caché
Creación de pedidos	≈ 5 pedidos/min en punta	No: transacción corta en HTTP/JSON
Reporte de ventas	Agregado diario/semanal sobre unos miles de pedidos	No requiere big data: aggregation de Mongo con índice (estado, creado_en)
Eventos de despacho en tiempo real	No exigidos por el caso	No

9.1 HTTP / JSON

Contexto, tráfico bajo, mensajes pequeños, sin streaming y frontend y GraphQL ya hablan HTTP / JSON

Alternativas, primera HTTP / JSON, simple y depurables. Segunda gRPC entre servicios, mensajes tipados y streaming pero requiere mTLS y más pruebas. Tercera un servicio de analítica separado, que se justifica solo con el volumen alto.

Decisión, usar HTTP / JSON y resolver los reportes dentro de orders con una consulta agregada que excluye datos personales.

Consecuencia, menos complejidad. Si la prueba de carga supera los objetivos se reevalúa gRPC solo para orders

9.2 Prueba de carga que respalda la decisión

Parámetro	**Valor**
Herramienta	k6 (o Locust)
Entorno / datos	docker-compose local; 10.000 pedidos y 200 productos ficticios
Escenario	50 usuarios simultáneos durante 5 min: 80 % GET catálogo, 15 % GET pedido, 5 % POST pedido
Objetivos	p95 < 300 ms en catálogo; errores < 1 %; reporte S21 < 1 s
Se registra	p95, throughput, % errores, CPU y memoria por contenedor (docker stats)

Con estos resultados se verifica que la alternativa implementada cumple los objetivos de latencia y errores declarados para la carga prevista. Si en el futuro el volumen creciera y la comunicación entre los orders e inventory se volviera un cuello de botella, gRPC podría incorporarse solo en ese tramo.

Escenario de prueba	Concurrencia	Solicitudes totales	Rendimiento (Req/s)	p50 (ms)	p95 (ms)	p99 (ms)	Errores (%)	Objetivo	Estado
Catálogo (`GET /products`) — Saturación	20	1.530	75,6	274,5	348,5	376,9	0%	≥ 50 req/s	Cumple
Catálogo (`GET /products`) — Carga fija	50 req/s	1.000	50,0	15,4	53,8	76,5	0%	P95 ≤ 200$ ms	Cumple
Crear pedido (`POST /orders`) — Saturación	10	618	30,7	326,1	416,7	460,5	0%	≥5 req/s	Cumple
Crear pedido (`POST /orders`) — Carga fija	5 req/s	100	5,0	52,4	83,9	93,8	0%	P95 ≤ 500 ms	Cumple
gRPC `PublishOrderEvent` — Saturación	8	17.231	860,1	2,9	36,1	130,8	0%	P95 ≤ 50 ms	Cumple
gRPC `GetSalesSummary` (30 días)	4	612	30,5	133,8	175,4	205,0	0%	P95 ≤ 300 ms	Cumple
gRPC `GetSalesSummary` (366 días)	4	81	4,0	1.012,0	1.096,4	1.137,9	0%	P95 ≤ 1000 ms	NO cumple

Modulo 10. Código fuente, persistencia, despliegue reproducible, integración completa, documentación y pruebas ejecutadas

10.1 descripción de la arquitectura e integración

El sistema se compone de una arquitectura de microservicios desacoplados basados en FastAPI (Python) y node.js (GraphQL), utilizando MongoDB como capa de persistencia. La comunicación externa se realiza exclusivamente a través del API Gateway (puerto 8000), el cual aplica políticas de enrutamiento, rate-limiting e inyección de la cabecera secreta (x-gateway-secret) para autorizar llamadas a la red privada

Componentes del sistema:

- API Gateway: Puerto 8000 (Python / FastAPI)

- Auth Service: Puerto 8001 (Python / FastAPI)

- Inventory Service: Puerto 8002 (Python / FastAPI)

- Orders Service: Puerto 8003 (Python / FastAPI)

- GraphQL Gateway: Puerto 8090 (Node.js / Express / Apollo)

- Persistencia: MongoDB en mongodb://localhost:27017 (Base de datos: `pizzeria`)

- Frontend: Servidor Apache/XAMPP o PHP Embebido en puerto 8080

10.2 Requisitos previos e instalación de dependencias

Asegurase de contar con python 3.10+, node.js 18+ y mongoDB community servido.

Entorno python:

```
cd "proyecto 2"
python –m venv .venv
.\.venv\Scripts\Activate.ps1
pip install fastapi uvicorn motor pydantic httpx bcrypt pyjwt
```

Entorno Node.js (GraphQL Gateway):

```
npm install apollo-server-express express graphql axios
```

10.3 secuencia de despliegue reproducible

Abre 5 ventanas de terminal independientes en el directorio raíz de la aplicación backend (`proyecto 2`):

[Terminal 1] Servicio de Autenticación (Auth Service - Puerto 8001)

```
cd "proyecto 2"
.\.venv\Scripts\Activate.ps1
python -m uvicorn auth_service:app --port 8001 --reload
```

[Terminal 2] Servicio de Inventario y Catálogo (Inventory Service - Puerto 8002)

```
cd "proyecto 2"
.\.venv\Scripts\Activate.ps1
python -m uvicorn inventory_service:app --port 8002 --reload
```

[Terminal 3] Servicio de Órdenes y Contactos (Orders Service - Puerto 8003)

```
cd "proyecto 2"
.\.venv\Scripts\Activate.ps1
python -m uvicorn orders_service:app --port 8003 --reload
```

[Terminal 4] API Gateway Principal (Gateway Proxy - Puerto 8000)

```
cd "proyecto 2"
.\.venv\Scripts\Activate.ps1
python -m uvicorn gateway:app --port 8000 --reload
```

[Terminal 5] GraphQL Gateway de Agregación (GraphQL Engine - Puerto 8090)

```
cd "proyecto 2"
node graphql_gateway.js
```

10.4 poblamiento inicial de datos ficticios de pruebas

Para inicializar las colecciones en MongoDB Compass (`pizzeria.pizzas` y `pizzeria.combos`), ejecuta el siguiente script interactivo de carga de datos:

Comando:

python seed_database.py

Contenido del Script `seed_database.py`:

```
import asyncio

from motor.motor_asyncio import AsyncIOMotorClient

async def seed():

client = AsyncIOMotorClient("mongodb://localhost:27017")

db = client["pizzeria"]

# Limpiar y reinsertar catálogo

await db.pizzas.delete_many({})

await db.combos.delete_many({})

pizzas = [

{"nombre": "Pizza Margherita", "ingredientes": ["Salsa Tomate", "Queso Mozzarella", "Albahaca"], "precio": 8500, "imagen": "img/margherita.jpg"},

{"nombre": "Pizza Pepperoni", "ingredientes": ["Salsa Tomate", "Mozzarella", "Pepperoni"], "precio": 9500, "imagen": "img/pepperoni.jpg"},

{"nombre": "Pizza Vegetariana", "ingredientes": ["Salsa Blanca", "Champiñones", "Pimentón"], "precio": 9000, "imagen": "img/veggie.jpg"}

]

combos = [

{"nombre": "Pack Dupla Student", "descripcion": "2 Pizzas Familiares + Bebida 1.5L", "precio": 17500, "imagen": "img/combo1.jpg"},

{"nombre": "Promo Casona UNAB", "descripcion": "1 Pizza Personalizada + 2 Jugos Naturales", "precio": 12000, "imagen": "img/combo2.jpg"}

]
```

```
await db.pizzas.insert_many(pizzas)

await db.combos.insert_many(combos)

print("¡Base de datos MongoDB poblada exitosamente!")

client.close()

if __name__ == "__main__":

asyncio.run(seed())
```

10.5 verificacion de estado y health checks

Comprueba el estado operativo de cada microservicio en el navegador:

- Gateway Healthcheck: GET http://localhost:8000/docs
- Inventory API: GET http://localhost:8000/api/v1/inventory/items
- GraphQL Sandbox: POST http://localhost:8090/graphql

10.6 suite de pruebas de integración automatizadas

Ejecuta el script de pruebas unitarias y de integración para validar el aislamiento de red, el cálculo de precios y la persistencia en MongoDB:

Comando:

python test_integration.py
Contenido del Script `test_integration.py`:

```
import httpx

import pytest

GATEWAY_URL = "http://localhost:8000/api/v1"

DIRECT_ORDERS_URL = "http://localhost:8003/orders"

def test_01_acceso_directo_rechazado():

# Intento de bypass del Gateway sin cabecera X-Gateway-Secret

resp = httpx.post(DIRECT_ORDERS_URL, json={"items": []})

assert resp.status_code == 403

print("Prueba 1 Aprobada: Acceso directo no autorizado rechazado (403).")
```

```
def test_02_creacion_pedido_dinamico():

payload = {

"items": [

{"personalizada": {"tamano": "mediana", "salsa": "bbq", "extras": ["Pepperoni",
"Tocino"]}, "precio": 9000, "cantidad": 1}

],

"tipo_entrega": "despacho",

"direccion": "Av. República 239",

"local_id": "casona"

}

resp = httpx.post(f"{GATEWAY_URL}/orders/orders", json=payload)

assert resp.status_code == 201

data = resp.json()

assert data["total"] == 9000

assert data["estado"] == "PENDIENTE_PAGO"

print(f"Prueba 2 Aprobada: Orden registrada con ID {data['id']} y Total
${data['total']}.")

if __name__ == "__main__":

test_01_acceso_directo_rechazado()

test_02_creacion_pedido_dinamico()
```

10.7 matriz de requisitos, servicios, pruebas y evidencias

requisito	caso/accion	Servicio backend	Prueba realizada	Evidencia verificable
R01 Catálogo	Lectura menu	S01 Inventory	GET `/api/v1/inventory/items`	Respuesta JSON 200 con listado de pizzas en ejecutor HTTP / Swagger.
R02 Personaliz.	Recalculo de pedido	S08 orders	POST `/orders` via script	Recálculo exacto: Base $7.000 + 2 extras = $9.000 (validación por script).

R03 Persist.	Finalizar compra	S08 / MongoDB	Insert `db.orders`	Registro visible en MongoDB Compass con ID único.
R04 Seguridad	Direct access	API Gateways	POST `:8003` sin header	Retorno HTTP 403 Forbidden por falta de clave secreta (`test_integration.py`).
R05 Agregación	Resumen de pedido	GraphQL Gatewa	Query GraphQL	Agregación de catálogo y pedidos comprobada en Sandbox (puerto 8090).
R06 Datos Din.	Datos de compra	Orders / Clients	Script interactivo / Payload REST.js	Dirección ("Av. República") y Local ("Casona") persisten en MongoDB.

10.8 losg de sistema e identificadores de correlacion

Todos lós eventos de log generados por los servicios incluyen un token de correlación simbólico (X-Correlation-ID) para trazabilidad en depuración:

Ejemplo de Log de Producción:

[2026-11-09 21:04:12] [INFO] [X-Correlation-ID: corr-8a9f-4b12] Gateway routing -> /orders/orders
[2026-11-09 21:04:12] [INFO] [X-Correlation-ID: corr-8a9f-4b12] Orders Service: Total recalculado = $17.500 CLP. Documento persistido en MongoDB.